SQL FOR DATA ANALYSIS · LESSON 20 OF 20

SQL Across Platforms: SQL Server, BigQuery, Snowflake and Databricks SQL

Learn the same analysis on the four platforms most job postings name, see where their SQL dialects differ (dates, string functions, limits, top-N, identifiers), and know which platform concept maps to which.

Gap-fill lesson | ✦ 2026 | Advanced | ⏱ 120 min

By the end of this lesson you can

- Name the main SQL platforms and what each is typically used for
- Translate common operations (top N, dates, string concatenation, casting) between dialects
- Explain the platform concepts of warehouse, dataset or schema and compute cost
- Write portable SQL and know which parts to rewrite when moving platforms

Datasets used

`superstore_sales.csv`

One language, four dialects

SQL is a standard, but every platform adds its own functions, limits and naming rules. An analyst who knows the logic can move between platforms quickly; the work is in translating the syntax. The queries in this course run in DuckDB, which is close to PostgreSQL. The examples below show the same question written for each platform.

Platform	Typical use	Where the company runs it
SQL Server (T-SQL)	Operational databases, enterprise reporting	On-premises, or Azure SQL
BigQuery	Serverless analytics on very large data	Google Cloud
Snowflake	Cloud data warehouse, shared data	Snowflake on AWS, Azure or Google Cloud
Databricks SQL	SQL on a lakehouse (Delta tables)	Databricks on the major clouds

CASE STUDY **One question, four platforms**

A product manager asks for the top five cities by profit in 2025. The analyst writes the same logic on each platform, because the company's warehouse, reporting tool and data team may each use a different one.

Top N rows

The question "top five cities by profit" is written differently on each platform.

SQL

```sql
-- SQL Server (T-SQL)
SELECT TOP 5 city, SUM(profit) AS profit
FROM superstore
WHERE YEAR(order_date) = 2025
GROUP BY city
ORDER BY profit DESC;

-- BigQuery
SELECT city, SUM(profit) AS profit
FROM `project.dataset.superstore`
WHERE EXTRACT(YEAR FROM order_date) = 2025
GROUP BY city
ORDER BY profit DESC
LIMIT 5;

-- Snowflake
SELECT city, SUM(profit) AS profit
FROM superstore
WHERE YEAR(order_date) = 2025
GROUP BY city
ORDER BY profit DESC
LIMIT 5;

-- Databricks SQL (Spark SQL)
SELECT city, SUM(profit) AS profit
FROM superstore
WHERE YEAR(order_date) = 2025
GROUP BY city
ORDER BY profit DESC
LIMIT 5;
```

The logic is identical. The differences are `TOP` against `LIMIT`, and the way each platform names tables and the date parts it accepts.

The same query runs in DuckDB, which is the engine used for the lessons in this course:

SQL

```sql
SELECT city, ROUND(SUM(profit), 0) AS profit
FROM superstore
WHERE EXTRACT(YEAR FROM order_date) = 2025
GROUP BY city
ORDER BY profit DESC
LIMIT 5;
```

RESULT

city	profit
Pune	912,526
Kolkata	739,621
Indore	527,368
Bengaluru	527,164
Chennai	504,760

5 rows × 2 columns

Dates

Date handling is the most common source of dialect errors.

Task	SQL Server	BigQuery	Snowflake	Databricks SQL
Year of a date	`YEAR(d)`	`EXTRACT(YEAR FROM d)`	`YEAR(d)`	`YEAR(d)`
Start of month	`DATEFROMPARTS(YEAR(d), MONTH(d), 1)`	`DATE_TRUNC(d, MONTH)`	`DATE_TRUNC('month', d)`	`DATE_TRUNC('MONTH', d)`
Add 30 days	`DATEADD(day, 30, d)`	`DATE_ADD(d, INTERVAL 30 DAY)`	`DATEADD(day, 30, d)`	`DATE_ADD(d, 30)`
Days between	`DATEDIFF(day, a, b)`	`DATE_DIFF(b, a, DAY)`	`DATEDIFF(day, a, b)`	`DATEDIFF(day, a, b)`

Notice that `DATEDIFF` takes its arguments in a different order on BigQuery: `DATE_DIFF(later, earlier, DAY)`. Argument order is a frequent source of wrong results that pass without error.

Strings and casting

```sql
-- SQL Server: concatenation with +, and NULL makes the result NULL
SELECT customer_name + ' (' + city + ')' FROM customers;

-- Snowflake, BigQuery and Databricks: || or CONCAT
SELECT customer_name || ' (' || city || ')' FROM customers;
SELECT CONCAT(customer_name, ' (', city, ')') FROM customers;

-- Casting
-- SQL Server and Snowflake: CAST(x AS INT)
-- BigQuery: CAST(x AS INT64)
-- Databricks: CAST(x AS INT)
```

Concatenation with + in SQL Server returns NULL if any part is NULL, while `CONCAT` treats NULL as an empty string. Check your platform's behaviour before trusting names with missing middle parts.

Table names and identifiers

- **SQL Server**: `database.schema.table`, with square brackets for unusual names: `[Order Date]`.
- **BigQuery**: `project.dataset.table`, with backticks around the full name.
- **Snowflake**: `database.schema.table`. Unquoted names are converted to upper case, so `order_date` and `ORDER_DATE` are the same column unless quoted.
- **Databricks**: `catalog.schema.table` in Unity Catalog, or `schema.table` in older workspaces.

::::warn Case sensitivity Snowflake folds unquoted names to upper case. A quoted name such as `"Order_Date"` is different from `order_date`. Choose one naming convention and stick to it. ::::

Platform concepts you will hear about

- **Warehouse (Snowflake)**: a compute cluster that you size and pause. Cost depends on how long it runs, so auto-suspend matters.
- **Slots and bytes billed (BigQuery)**: you pay for the bytes a query scans, so selecting only the columns you need reduces cost.
- **Delta tables (Databricks)**: tables stored as Parquet files with a transaction log. They support `MERGE`, time travel and schema evolution, covered in the Power BI and Fabric lessons.
- **Clusters and SQL warehouses (Databricks)**: compute attached to notebooks or SQL endpoints.
- **Indexes and statistics (SQL Server)**: a clustered index orders the table on disk. Columnar warehouses such as Snowflake and BigQuery handle scans differently and rely less on manual indexes.

Writing portable SQL

When the company may move platforms, keep the logic in a layer that is easy to translate:

1. Use ANSI functions where they exist: `CASE`, `COALESCE`, `CAST`, `SUM OVER`, `ROW_NUMBER`, `DATE_TRUNC` (with care about its argument order on each platform).
2. Keep platform-specific date and string code in views or macros, not scattered through reports.
3. Use the same test queries on every platform, such as row counts and totals per month, to check a migration.
4. Note each platform's limits, such as the maximum number of columns in a select or the size of a `GROUP BY`.

::::tip Test the translation with numbers A moved query is correct only when it returns the same totals. Compare the old and new results by month and by category, not just by the grand total, because two errors can cancel out in the total. ::::

::::key What to remember - The logic of SQL transfers; the dialect (top N, dates, strings, identifiers) does not. - Check argument order for date functions on every platform. - Snowflake folds unquoted names to upper case; BigQuery uses backticks and three-part names. - Compute cost is a platform concept: warehouses (Snowflake), bytes scanned (BigQuery) and clusters (Databricks). - Verify any migration by comparing results by period and category. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which clause returns the first five rows in SQL Server?

A LIMIT 5

B TOP 5

C FETCH FIRST 5 ROWS ONLY on every platform

D ROWNUM < 5 only

Answer: B. SQL Server uses TOP; LIMIT is used by BigQuery, Snowflake and Databricks.

2. BigQuery's DATE_DIFF takes its arguments in which order?

A Earlier date then later date

B Later date then earlier date, then the unit

C The unit first

D Dates are not allowed

Answer: B. DATE_DIFF(later, earlier, DAY) gives a positive result for a later first date.

3. In SQL Server, `'Asha' + ' ' + NULL` returns:

A 'Asha '

B NULL

C 'Asha'

D An error only

Answer: B. SQL Server's + operator returns NULL when any part is NULL.

4. Snowflake folds unquoted identifiers to:

A Lower case

B Upper case

C Title case

D Their original case

Answer: B. `order_date` and `ORDER_DATE` are the same column unless quoted.

5. On BigQuery, the most direct way to reduce cost is to:

A Add more indexes

B Select only the columns the query needs

C Turn off caching

D Use more warehouses

Answer: B. BigQuery bills by the bytes scanned.

EXERCISE **Translate a query**

Rewrite this T-SQL for Snowflake and for BigQuery, keeping the same result: `SELECT TOP 10 customer_id, DATEDIFF(day, MIN(order_date), MAX(order_date)) AS days_active FROM orders GROUP BY customer_id ORDER BY days_active DESC;`

Show solution (worked answer)

Snowflake: the same `DATEDIFF(day, ...)` works, with `LIMIT 10` replacing `TOP 10`. BigQuery: `DATE_DIFF(MAX(order_date), MIN(order_date), DAY)` (later date first), `LIMIT 10` at the end, and the table written as `project.dataset.orders`.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.